JavaScript is disabled on your browser.

compact1, compact2, compact3

java.time.chrono

Class AbstractChronology

java.lang.Object
  java.time.chrono.AbstractChronology

All Implemented Interfaces:

`Comparable<Chronology>, Chronology`

Direct Known Subclasses:

`HijrahChronology, IsoChronology, JapaneseChronology, MinguoChronology, ThaiBuddhistChronology`

```
public abstract class AbstractChronology
extends Object
implements Chronology
```

An abstract implementation of a calendar system, used to organize and identify dates.

The main date and time API is built on the ISO calendar system. The chronology operates behind the scenes to represent the general concept of a calendar system.

See `Chronology` for more details.

Implementation Requirements:

This class is separated from the Chronology interface so that the static methods are not inherited. While Chronology can be implemented directly, it is strongly recommended to extend this abstract class instead.

This class must be implemented with care to ensure other classes operate correctly. All implementations that can be instantiated must be final, immutable and thread-safe. Subclasses should be Serializable wherever possible.

Since:

1.8

Constructor Summary

Constructors

Modifier	Constructor	Description
protected	`AbstractChronology()`	Creates an instance.

Method Summary

Concrete Methods

Modifier and Type	Method	Description
int	`compareTo(Chronology other)`	Compares this chronology to another chronology.
boolean	`equals(Object obj)`	Checks if this chronology is equal to another chronology.
int	`hashCode()`	A hash code for this chronology.
`ChronoLocalDate`	`resolveDate(Map<TemporalField,Long> fieldValues, ResolverStyle resolverStyle)`	Resolves parsed `ChronoField` values into a date during parsing.

String	toString()	Outputs this chronology as a String, using the chronology ID.

Methods inherited from class java.lang.Object

clone, finalize, getClass, notify, notifyAll, wait, wait, wait

Methods inherited from interface java.time.chrono.Chronology

date, date, date, dateEpochDay, dateNow, dateNow, dateNow, dateYearDay, dateYearDay, eraOf, eras, from, getCalendarType, getDisplayName, getId, isLeapYear, localDateTime, period, prolepticYear, range, zonedDateTime, zonedDateTime

Constructor Detail

AbstractChronology

```
protected AbstractChronology()
```

Creates an instance.

Method Detail

resolveDate

```
public ChronoLocalDate resolveDate(Map<TemporalField,Long> fieldValues,
                                   ResolverStyle resolverStyle)
```

Resolves parsed ChronoField values into a date during parsing.

Most TemporalField implementations are resolved using the resolve method on the field. By contrast, the ChronoField class defines fields that only have meaning relative to the chronology. As such, ChronoField date fields are resolved here in the context of a specific chronology.

`ChronoField` instances are resolved by this method, which may be overridden in subclasses.

- `EPOCH_DAY` - If present, this is converted to a date and all other date fields are then cross-checked against the date.
- `PROLEPTIC_MONTH` - If present, then it is split into the `YEAR` and `MONTH_OF_YEAR`.
  If the mode is strict or smart then the field is validated.
- `YEAR_OF_ERA` and `ERA` - If both are present, then they are combined to form a `YEAR`. In lenient mode, the `YEAR_OF_ERA` range is not validated, in smart and strict mode it is. The `ERA` is validated for range in all three modes. If only the `YEAR_OF_ERA` is present, and the mode is smart or lenient, then the last available era is assumed. In strict mode, no era is assumed and the `YEAR_OF_ERA` is left untouched. If only the `ERA` is present, then it is left untouched.
- `YEAR`, `MONTH_OF_YEAR` and `DAY_OF_MONTH` - If all three are present, then they are combined to form a date. In all three modes, the `YEAR` is validated. If the mode is smart or strict, then the month and day are validated. If the mode is lenient, then the date is combined in a manner equivalent to creating a date on the first day of the first month in the requested year, then adding the difference in months, then the difference in days. If the mode is smart, and the day-of-month is greater than the maximum for the year-month, then the day-of-month is adjusted to the last day-of-month. If the mode is strict, then the three fields must form a valid date.
- `YEAR` and `DAY_OF_YEAR` - If both are present, then they are combined to form a date. In all three modes, the `YEAR` is validated. If the mode is lenient, then the date is combined in a manner equivalent to creating a date on the first day of the requested year, then adding the difference in days. If the mode is smart or strict, then the two fields must form a valid date.
- `YEAR`, `MONTH_OF_YEAR`, `ALIGNED_WEEK_OF_MONTH` and `ALIGNED_DAY_OF_WEEK_IN_MONTH` - If all four are present, then they are combined to form a date. In all three modes, the `YEAR` is validated. If the mode is lenient, then the date is combined in a manner equivalent to creating a date on the first day of the first month in the requested year, then adding the difference in months, then the difference in weeks, then in days. If the mode is smart or strict, then the all four fields are validated to their outer ranges. The date is then combined in a manner equivalent to creating a date on the first day of the requested year and month, then adding the amount in weeks and days to reach their values. If the mode is strict, the date is additionally validated to check that the day and week adjustment did not change the month.
- `YEAR`, `MONTH_OF_YEAR`, `ALIGNED_WEEK_OF_MONTH` and `DAY_OF_WEEK` - If all four are present, then they are combined to form a date. The approach is the same as described above for years, months and weeks in `ALIGNED_DAY_OF_WEEK_IN_MONTH`. The day-of-week is adjusted as the next or same matching day-of-week once the years, months and weeks have been handled.

- YEAR, ALIGNED_WEEK_OF_YEAR and ALIGNED_DAY_OF_WEEK_IN_YEAR - If all three are present, then they are combined to form a date. In all three modes, the YEAR is validated. If the mode is lenient, then the date is combined in a manner equivalent to creating a date on the first day of the requested year, then adding the difference in weeks, then in days. If the mode is smart or strict, then the all three fields are validated to their outer ranges. The date is then combined in a manner equivalent to creating a date on the first day of the requested year, then adding the amount in weeks and days to reach their values. If the mode is strict, the date is additionally validated to check that the day and week adjustment did not change the year.
- YEAR, ALIGNED_WEEK_OF_YEAR and DAY_OF_WEEK - If all three are present, then they are combined to form a date. The approach is the same as described above for years and weeks in ALIGNED_DAY_OF_WEEK_IN_YEAR. The day-of-week is adjusted as the next or same matching day-of-week once the years and weeks have been handled.

The default implementation is suitable for most calendar systems. If ChronoField.YEAR_OF_ERA is found without an ChronoField.ERA then the last era in Chronology.eras() is used. The implementation assumes a 7 day week, that the first day-of-month has the value 1, that first day-of-year has the value 1, and that the first of the month and year always exists.

Specified by:

resolveDate in interface Chronology

Parameters:

fieldValues - the map of fields to values, which can be updated, not null

resolverStyle - the requested type of resolve, not null

Returns:

the resolved date, null if insufficient information to create a date

Throws:

DateTimeException - if the date cannot be resolved, typically because of a conflict in the input data

compareTo

```
public int compareTo(Chronology other)
```

Compares this chronology to another chronology.

The comparison order first by the chronology ID string, then by any additional information specific to the subclass. It is "consistent with equals", as defined by Comparable.

Specified by:

`compareTo` in interface `Comparable<Chronology>`

Specified by:

`compareTo` in interface `Chronology`

Implementation Requirements:

This implementation compares the chronology ID. Subclasses must compare any additional state that they store.

Parameters:

`other` - the other chronology to compare to, not null

Returns:

the comparator value, negative if less, positive if greater

equals

```
public boolean equals(Object obj)
```

Checks if this chronology is equal to another chronology.

The comparison is based on the entire state of the object.

Specified by:

`equals` in interface `Chronology`

Overrides:

`equals` in class `Object`

Implementation Requirements:

This implementation checks the type and calls `compareTo(java.time.chrono.Chronology)`.

Parameters:

`obj` - the object to check, null returns false

Returns:

true if this is equal to the other chronology

See Also:

`Object.hashCode()`, `HashMap`

hashCode

```
public int hashCode()
```

A hash code for this chronology.

The hash code should be based on the entire state of the object.

Specified by:

`hashCode` in interface `Chronology`

Overrides:

`hashCode` in class `Object`

Implementation Requirements:

This implementation is based on the chronology ID and class. Subclasses should add any additional state that they store.

Returns:

a suitable hash code

See Also:

`Object.equals(java.lang.Object)`, `System.identityHashCode(java.lang.Object)`

toString

```
public String toString()
```

Outputs this chronology as a `String`, using the chronology ID.

Specified by:

`toString` in interface `Chronology`

Overrides:

`toString` in class `Object`

Returns:

a string representation of this chronology, not null

Copyright © 1993, 2026, Oracle and/or its affiliates. All rights reserved. Use is subject to license terms.
Also see the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.